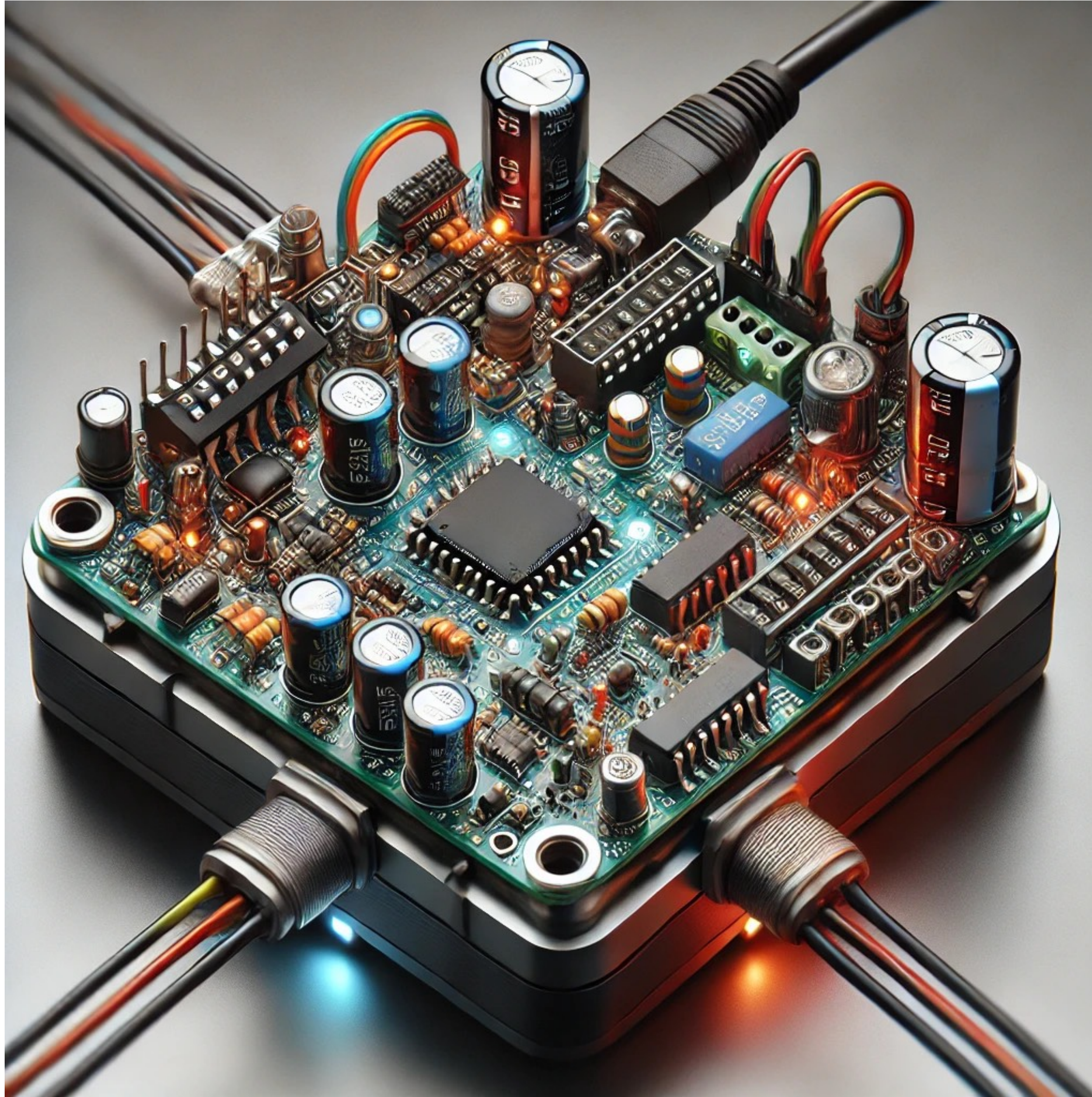


Embedded Development and Debugging

Managing Linux With the Embedded Perspective **TX00EX85-3001**

1.09.2024

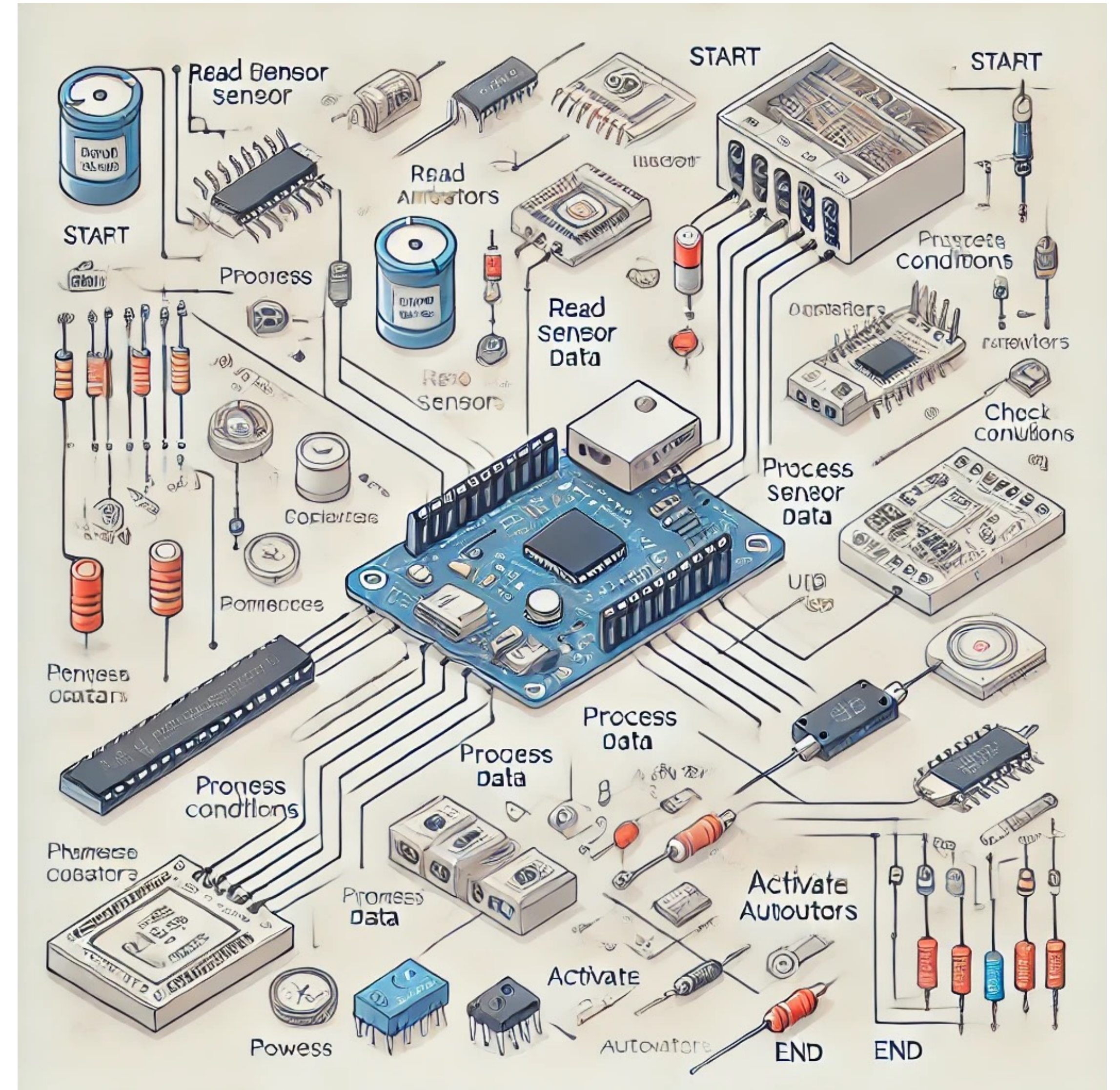


Embedded Systems Development

- Requirements/specifications
 - Functionality
 - Performance
 - Real-time requirements
 - Hardware limitations
- Standards and requirements

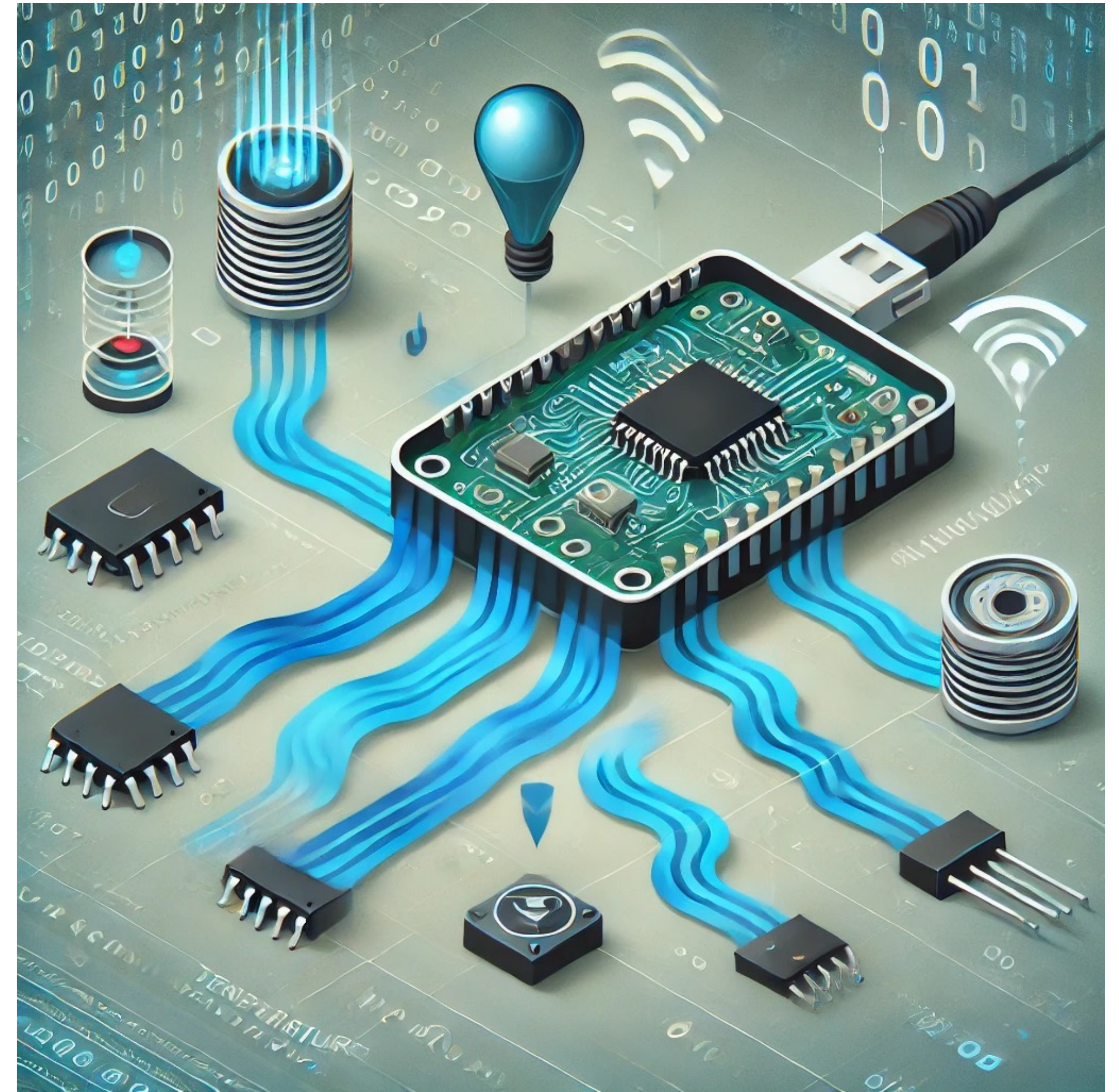
Design

- Architecture
 - hardware
 - μC
 - HW components
 - Peripherals
- Software architecture
- Protocols



Software Development

- Coding
 - Algorithms
 - protocols
 - C/C++ or Assembly (Sometimes)

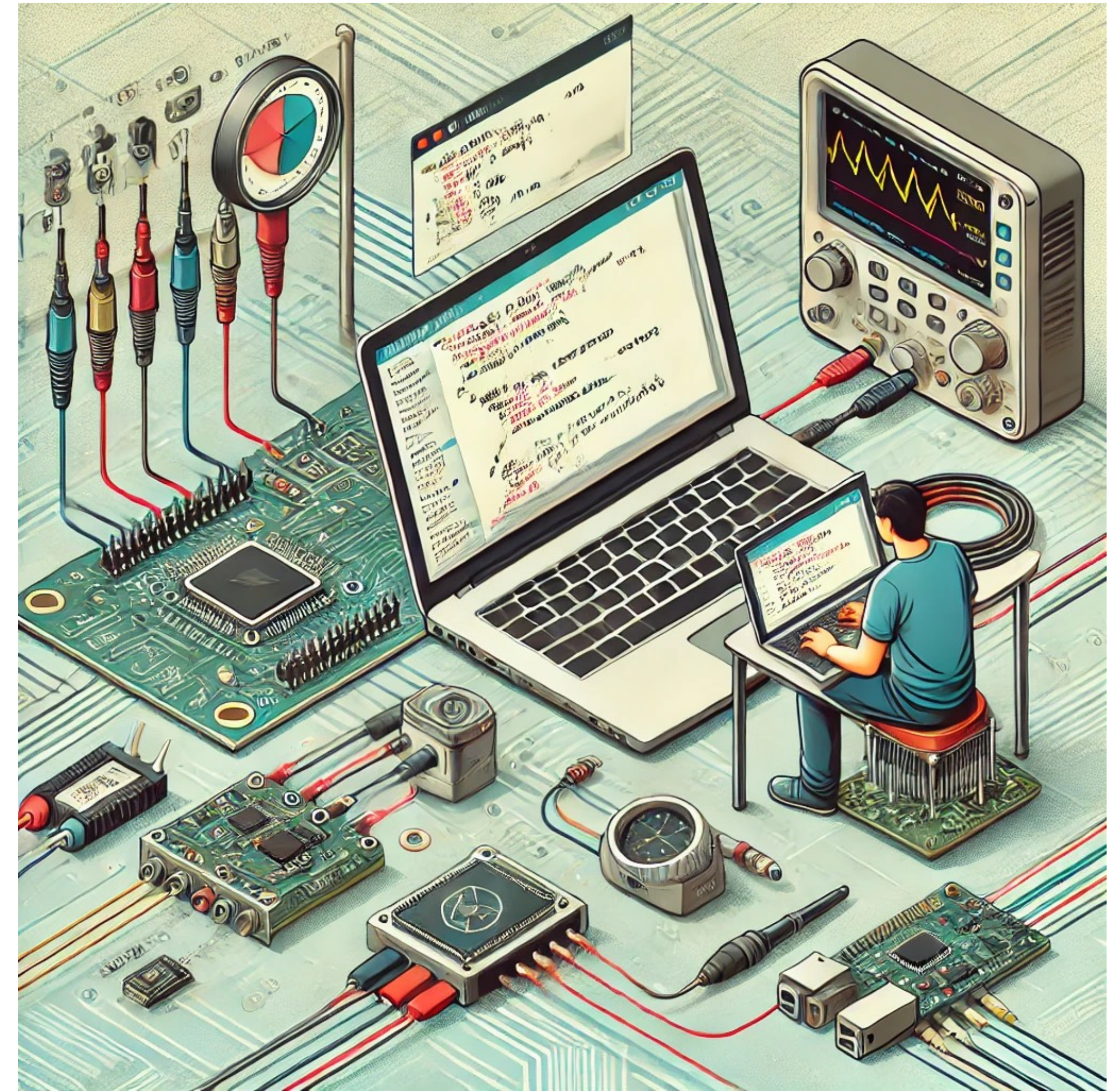


System Integration

- Connecting software and hardware
 - Checking the interactions between
 - μC
 - Peripherals
 - Sensors
 - Actuators etc.
-

Testing and Debugging

- Verifying hardware/software functionality
- Verifying performance
- Isolating issues
- Test automation often used
 - CICD
- Separate hardware required
 - Programmer/Debugger



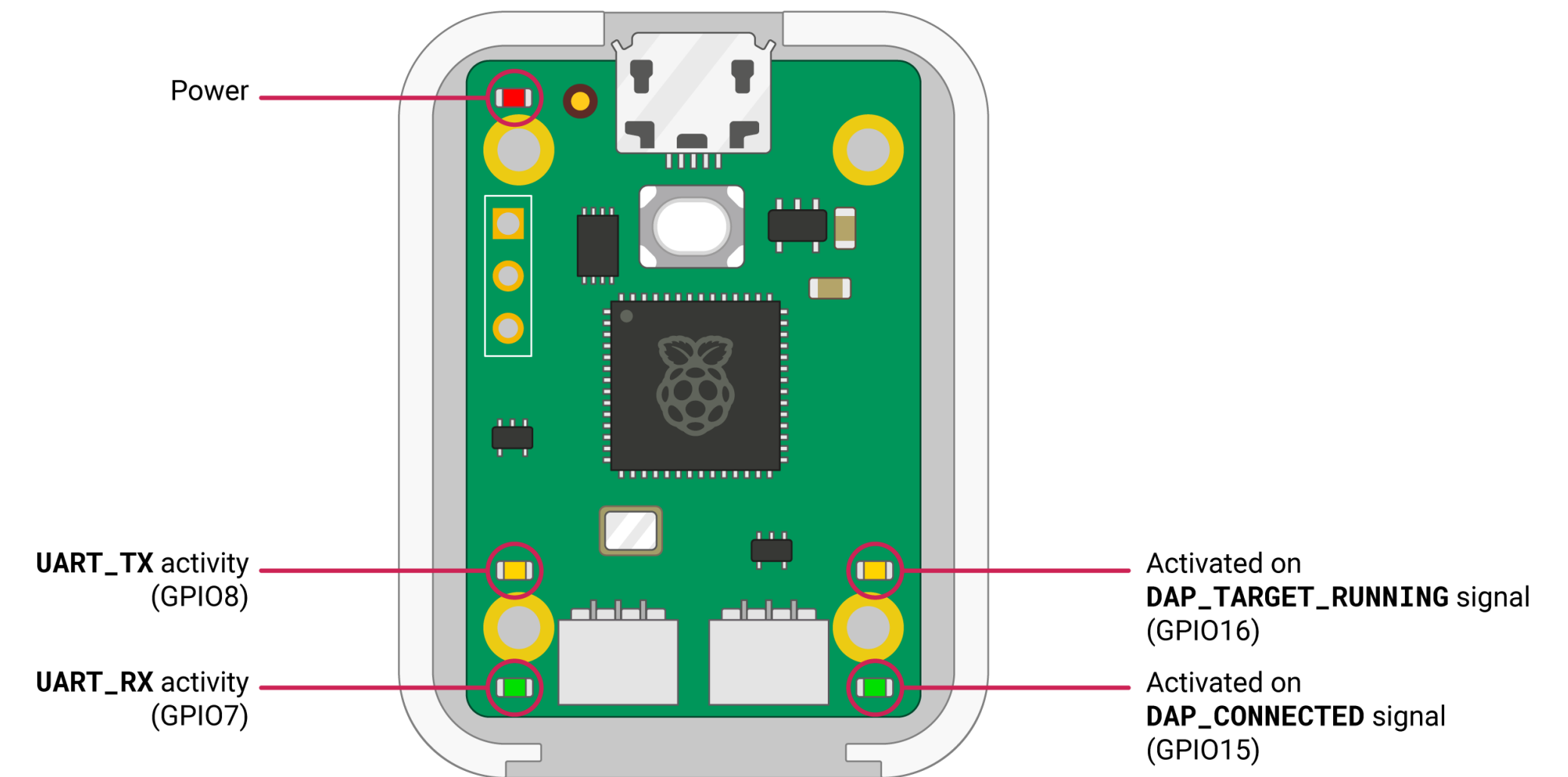
Optimize, Deploy, Maintain

- Improve performance, Memory usage, and power
- Flash devices
- Fix bugs, update firmware



Picoprobe debugger

- Hardware debugger for RP2040 based μ Cs
- UART interface
- SWD (Serial Wire Debug) interface
 - Used for flashing
 - Used to debug through DAP (Debug Access Port)
 - CMSIS-DAP standard debugging interface

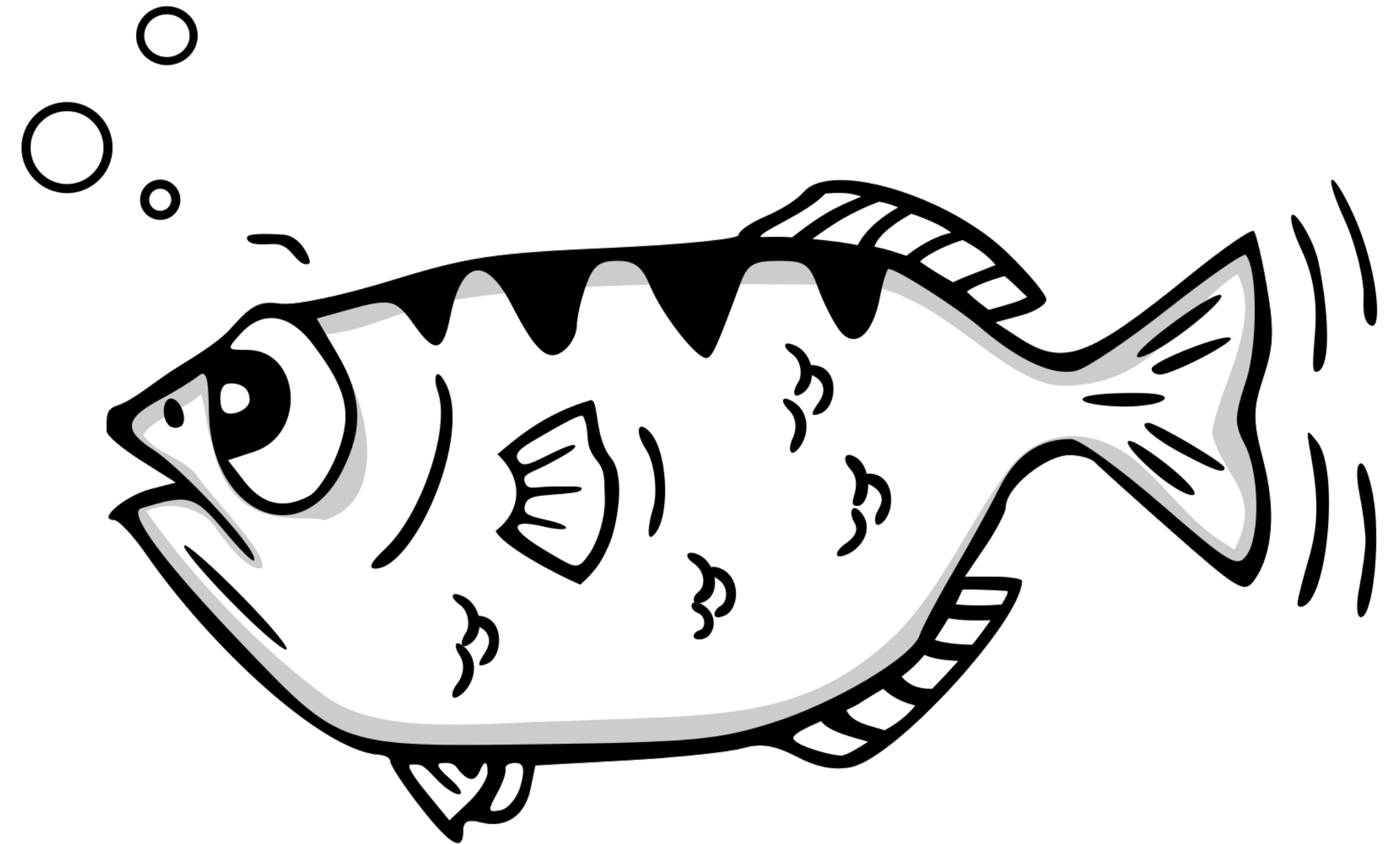


OpenOCD

- Open Source Software used for Debugging and Flashing
 - PC runs OpenOCD
 - Capable of running GDB server
 - Interfaces with μ Cs (CMSIS-DAP to Debug Pico)

GDB

- GNU Project Debugger
 - See what is going on inside your program at runtime.
 - Allows you to set breakpoints
 - Allows stepping
 - See what happens when program stops
 - Supports many languages, including C, C++, and Assembly



<https://www.sourceware.org/gdb/mascot/>

Installing OpenOCD and GDB

- Installing OpenOCD
 - Ubuntu
 - `sudo apt install -y openocd`
 - Installing GDB
 - Ubuntu
 - `sudo apt install -y gdb-multiarch`
-

Compiling for debugging

- Building your code for debugging with GDB
 - Must build code in Debug mode, so that the information needed for GDB is built into your .elf file
 - Create a debug folder in your CMake project and Change directories into it
 - Execute `cmake -DCMAKE_BUILD_TYPE=Debug ..` to have CMake generate you build files for debugging
 - Use make to build your code as per usual
-

Flashing your code with OpenOcd

- Using OpenOCD to flash your .elf is a single command
 - `sudo openocd -f <path to cmsis-dap.cfg> -f <path to target/rp2040.cfg> -c "adapter speed 5000" -c "program <name of your .elf file> verify reset exit"`
 - This will flash the binary to your pico using the PicoProbe and OpenOCD

Debugging

- Once the code is flashed to your pico, you can tell OpenOCD to start a GDB server instance
 - `sudo openocd -f <path to cmsis-dap.cfg> -f <path to target/rp2040.cfg> -c "adapter speed 5000"`
 - This will create a server that can be connected to on your local host at port 3333
 - In a separate terminal tab or window, you will start gdb-multiarch and establish a connection to your gdb server
 - `gdb-multiarch <name of .elf file>`
 - `target remote localhost:3333`
 - `monitor reset init`
 - `continue`
 - Now your firmware should be running and you should have a GDB terminal
-

Debugging in GDB

- Useful Commands
 - list - shows source code (Lines that you wish to view can be specified)
 - break - Sets a breakpoint (Can be set on a particular line or on a function of specified file)
 - info - gives info on different things (local, break, watchpoints.. etc)
 - print - can be used to print and evaluate expressions
 - kill - stop running but stay in GDB, when you reconnect to the server the program will run from the start with all breakpoints, watchpoints, etc still active
 - continue - run program until the next breakpoint or watch (If none set, the program just executes)
-

Debugging in GDB

- Useful Commands
 - next - step over to next line
 - step - steps into function
 - display - can be used to show the value of expressions whenever the execution is halted with a breakpoint
 - watch - halts execution on read or write of variable(mind scope)
-